Real PR reviews

- Same skill: `/dex-review-code`
- Review 1: *GENERIC review (NO domain knowledge)*
- Review 2: *DOMAIN-AWARE review (WITH domain knowledge)*
- Four PRs reviewed; we'll check two of them

#272038
Move install/upgrade/revert into `DetectionRulesClient`

#271722
`rulesClient.bulkCreate()`

#268165
Refine rules `_search`, `_review` API contracts

#269617
MVP UI for rule changes history

#272038 - Move install/upgrade/revert into `DetectionRulesClient`

🤔 GENERIC

`legacy_create_prepackaged_rules.ts`

...e/prebuilt_rules/api/install_prebuilt_rules_and_timelines/legacy_create_prepackaged_rules.ts

```
96        -     },
97        -   };
98        -   await upgradePrebuiltRules(detectionRulesClient, rulesToUpdate, upgradeChangeTracking,
      82  +   await applyPrebuiltRuleAssets({
```

banderror 4 days ago — Member

Inconsistency with PR goal: The upgrade step calls `applyPrebuiltRuleAssets` directly, bypassing `DetectionRulesClient`. The `detectionRulesClient` variable is in scope (used at R71 for `installAllPrebuiltRules`). Using `applyPrebuiltRuleAssets` here means upgrade operations in this legacy path do not go through the DRC's instrumentation/tracing wrappers, contrary to the PR's stated goal of consolidating prebuilt rule operations inside DRC.

Consider replacing lines R82–R85 with `await detectionRulesClient.upgradeAllPrebuiltRules(...)` or a dedicated upgrade call, so the legacy path is fully delegated to DRC.

🎯 DOMAIN-AWARE

`legacy_create_prepackaged_rules.ts`

...e/prebuilt_rules/api/install_prebuilt_rules_and_timelines/legacy_create_prepackaged_rules.ts

```
96        -     },
97        -   };
98        -   await upgradePrebuiltRules(detectionRulesClient, rulesToUpdate, upgradeChangeTracking,
      82  +   await applyPrebuiltRuleAssets({
```

banderror 4 days ago — Member

Inconsistency: install goes through DRC, upgrade bypasses it. The install step correctly delegates to `detectionRulesClient.installAllPrebuiltRules()` at line 71. But the upgrade step at lines 82–85 calls `applyPrebuiltRuleAssets` directly (a raw util method), bypassing the DRC's `upgradePrebuiltRules` / `upgradeAllPrebuiltRules` methods, which include revision checks, conflict resolution, and telemetry context collection.

If the intent is that `legacyCreatePrepackagedRules` is deprecated and should only minimally work, this should be documented. Otherwise, the upgrade step should also go through the DRC for consistency (`detectionRulesClient.upgradeAllPrebuiltRules(...)` or an equivalent).

#272038 - Move install/upgrade/revert into `DetectionRulesClient`

(no analog - generic review didn't find this issue)

🎯 **DOMAIN-AWARE**

`detection_rules_client_interface.ts`

`...tion_engine/rule_management/logic/detection_rules_client/detection_rules_client_interface.ts`

```
124 +   updatedRules: UpgradedRuleBasicInfo[];
125 +   skippedRules: SkippedRuleUpgrade[];
126 +   errors: Array<PromisePoolError<{ rule_id: string }>>;
127 +   ruleUpgradeContexts: Map<string, RuleUpgradeContext>;
```

banderror 4 days ago — Member

Domain invariant: abstraction-boundary leakage in public interface.

`UpgradePrebuiltRulesResult.ruleUpgradeContexts` exposes `Map<string, RuleUpgradeContext>` where `RuleUpgradeContext` is an internal implementation type from `prebuilt_rules/api/perform_rule_upgrade/update_rule_telemetry`. The `IDetectionRulesClient` interface is the abstraction boundary between callers and the rule-management domain; exposing a type from a sibling domain's API module in this interface blurs the boundary and creates an implicit coupling. The telemetry firing happens inside the route handler (`performRuleUpgradeHandler`) which calls `sendRuleUpdateTelemetryEvents` with the contexts map. If this type needs to cross the boundary, it should be re-exported from or defined in the `detection_rules_client_interface.ts` file itself (or a shared common module), not imported from a handler-level implementation file.

#271722 - `rulesClient.bulkCreate()`

🤔 GENERIC

route.ts

...ecurity_solution/server/lib/detection_engine/rule_management/api/rules/import_rules/route.ts

```
63  67              maxBytes: config.maxRuleImportPayloadBytes,
64  68              output: 'stream',
65  69            },
    70  +         tags: [routeLimitedConcurrencyTag(RULE_MANAGEMENT_IMPORT_CONCURRENCY)],
```

banderror 4 days ago — Member

`routeLimitedConcurrencyTag(1)` limits the entire import endpoint to one concurrent request cluster-wide.

This is a major throughput regression for concurrent users. The constant `RULE_MANAGEMENT_IMPORT_CONCURRENCY = 1` combined with `routeLimitedConcurrencyTag` means that while one user is importing rules, all other users' import requests will be queued (or dropped, depending on the tag semantics).

The comment on the constant says "mirrors `PREBUILT_RULES_OPERATION_CONCURRENCY`" — that constant limits concurrent *background* operations on a single node, not user-facing HTTP endpoints. For a user-facing API, limiting to 1 globally is extremely aggressive.

Please verify the intended semantics: is this intended to limit the number of simultaneous ES write batches, or the number of HTTP requests? If the former, consider moving the concurrency control inside the business logic (e.g. a semaphore in `bulkImportRules`). If the latter, document the intentional trade-off or raise the limit.

🎯 DOMAIN-AWARE

constants.ts

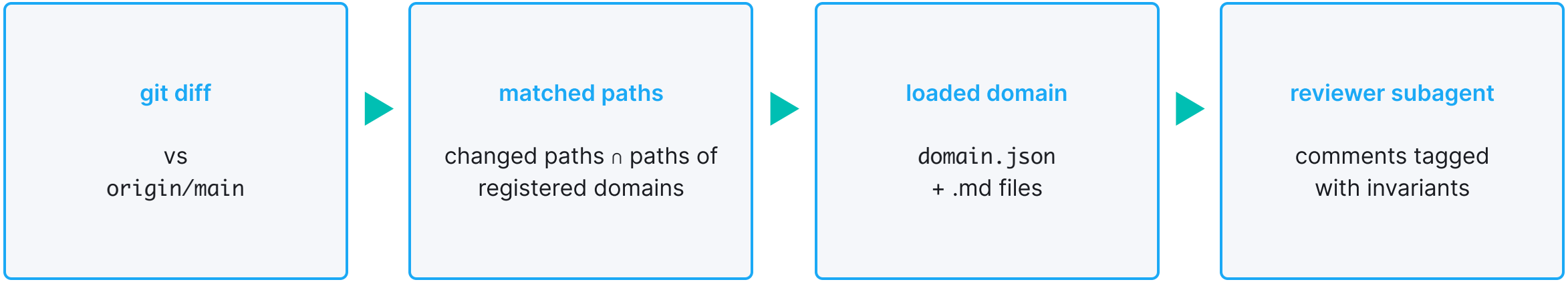
...urity/plugins/security_solution/server/lib/detection_engine/rule_management/api/constants.ts

```
15  15  export const RULE_MANAGEMENT_IMPORT_EXPORT_SOCKET_TIMEOUT_MS = 3600000 as const;
    16  +
    17  + /** Cap concurrent rule imports at 1 to bound heap; mirrors PREBUILT_RULES_OPERATION_CONC
    18  + export const RULE_MANAGEMENT_IMPORT_CONCURRENCY = 1;
```

banderror 4 days ago — Member

Domain invariant: heavy endpoints must rate-limit. Setting `RULE_MANAGEMENT_IMPORT_CONCURRENCY = 1` means only one import request can run at a time per Kibana node. The domain knowledge explicitly documents that the import route is one of the four hot paths (rank 2), and that MSSP customers run 300 spaces per cluster. With concurrency 1, a single long-running import on any space blocks all other concurrent imports on that node. The reference pattern (`bulk_actions/route.ts`) uses a named concurrency constant significantly higher than 1. If the intent is to bound heap during the new `bulkCreateRules`-based path, please document the reasoning (does the new path actually use more heap than the old one?) and consider whether a limit higher than 1 (e.g. 3–5) would still bound memory without completely serializing multi-space import workloads.

How domain knowledge discovery works



Anatomy of a domain knowledge file

DOMAIN.JSON

```
{
  "slug": "detection-rule-management",
  "name": "Detection Rule Management",
  "owners": ["@elastic/security-detection-rule-management"],
  "paths": [
    "x-pack/.../rule_management/**"
  ],
  "files": [
    { "path": "detection-rule-management.md" }
  ]
}
```

DETECTION-RULE-MANAGEMENT.MD

```
# Overview
   · What the domain is
# Architectural invariants
   · The boundaries that must not break
# Common review patterns
   · Recurring catches per area
# Security considerations
   · RBAC, space isolation, sanitization
# Performance constraints
   · Scale, throughput, latency
# Historical catches
   · Lessons from past PRs
```

Captured with `/dex-domain-capture`.

Smart is not the same as *informed*

> **The rules**: The model doesn't know that `IDetectionRulesClient` is **the** business logic abstraction in our domain.

> **The history**: It doesn't know we got burned in the past by exposing the data model via the API endpoints.

> **The scale**: It doesn't know that largest MSSP customers run 300 spaces × 3,000 rules per space, and need concurrent bulk actions across spaces.

What's in the context window is key

An AI agent is as good as the quality and accuracy of the information you put in its context window.

> **Domain knowledge**: today - missing, tomorrow - sufficient, eventually - comprehensive and accurate.

> **Team standards**: cross-domain rules and guidelines.

> **Decision log**: why certain decisions were made.